
Orientação a Objetos

Variáveis Primitivas e Controle de Fluxo (Loops)

Profa. Alana Neo

O WHILE

- O **while** é um comando usado para fazer um laço (**loop**), isto é, **repetir um trecho de código algumas vezes**.
- A ideia é que **esse trecho de código seja repetido enquanto uma determinada condição permanecer verdadeira**.

```
public class Loop {  
  
    public static void main(String[] args) {  
  
        int idade = 15;  
        while (idade < 18) {  
            System.out.println(idade);  
            idade = idade + 1;  
        }  
  
    }  
  
}
```

O WHILE

```
public class Loop {  
  
    public static void main(String[] args) {  
  
        int idade = 15;  
        while (idade < 18) {  
            System.out.println(idade);  
            idade = idade + 1;  
        }  
  
    }  
  
}
```

- O trecho dentro do bloco do **while** será executado até o momento em que a condição **idade < 18** passe a ser falsa.
- Isso ocorrerá exatamente no momento em que **idade == 18**, o que não o fará imprimir 18.

O WHILE

→ Exemplo:

```
public class Loop1 {  
  
    public static void main(String[] args) {  
  
        int i = 0;  
        while (i < 10) {  
            System.out.println(i);  
            i = i + 1;  
        }  
    }  
}
```

→ O que você acha que irá imprimir?

O WHILE

→ Exemplo:

```
public class Loop1 {  
    public static void main(String[] args) {  
        int i = 0;  
        while (i < 10) {  
            System.out.println(i);  
            i = i + 1;  
        }  
    }  
}
```

→ O que você acha que irá imprimir?

→ O **while** acima imprime de 0 a 9.

O FOR

- Outro comando de loop extremamente utilizado é o **for**.
- A ideia é a mesma do while: **fazer um trecho de código ser repetido enquanto uma condição continuar verdadeira.**
- Mas além disso, o **for** isola também um espaço para **inicialização de variáveis e o modificador dessas variáveis.**
- Isso faz com que fiquem mais legíveis, as variáveis que são relacionadas ao loop:

```
for (inicializacao; condicao; incremento) {  
    codigo;  
}
```

O FOR

→ Exemplo:

```
public class For {  
    public static void main(String[] args) {  
        for (int i = 0; i < 10; i = i + 1) {  
            System.out.println("olá!");  
        }  
    }  
}
```

O FOR

- Esse **for** poderia ser trocado por:

```
public class Usandowhile {  
  
    public static void main(String[] args) {  
  
        int i = 0;  
        while (i < 10) {  
            System.out.println("olá!");  
            i = i + 1;  
        }  
    }  
}
```

- Porém, o código do for indica claramente que a variável *i* serve, em especial, para controlar a quantidade de laços executados.
- Quando usar o for? Quando usar o while? Depende do gosto e da ocasião.

PRÉ e PÓS INCREMENTO ++

- $i = i + 1$ pode realmente ser substituído por $i++$ quando isolado, porém, em alguns casos, temos essa instrução envolvida em, por exemplo, uma atribuição:

```
public class Posincremento {  
  
    public static void main(String[] args) {  
  
        int i = 5;  
        int x = i++;  
        System.out.println(i);  
        System.out.println(x);  
  
    }  
  
}
```

→ Qual é o valor de x ?

→ Qual é o valor de i ?

PRÉ e PÓS INCREMENTO ++

- O de i , após essa linha, é 6.
- O operador ++ , quando vem após a variável, retorna o valor antigo, e incrementa (pós incremento), fazendo x valer 5.
- Usando o ++ antes da variável (pré incremento), o resultado é 6:

```
public class Preincremento {  
  
    public static void main(String[] args) {  
  
        int i = 5;  
        int x = ++i; // aqui x valera 6  
        System.out.println(i);  
        System.out.println(x);  
  
    }  
  
}
```

CONTROLANDO LOOPS

- Apesar de termos condições booleanas nos nossos laços, em algum momento, podemos decidir parar o loop por algum motivo especial sem que o resto do laço seja executado.

```
public class Break {  
    public static void main(String[] args) {  
        int x = 0;  
        int y = 30;  
        for (int i = x; i < y; i++) {  
            if (i % 19 == 0) {  
                System.out.println("Achei um número divisível por 19 entre x e y");  
                break;  
            }  
        }  
    }  
}
```

- O código acima vai percorrer os números de x a y e parar quando encontrar um número divisível por 19, uma vez que foi utilizada a palavra chave **break**.

CONTROLANDO LOOPS

- Da mesma maneira, é possível obrigar o loop a executar o próximo laço.
- Para isso usamos a palavra chave **continue**.

```
public class Continue {  
    public static void main(String[] args) {  
        for (int i = 0; i < 100; i++) {  
            if (i > 50 && i < 60) {  
                continue;  
            }  
            System.out.println(i);  
        }  
    }  
}
```

- O código acima não vai imprimir alguns números. (Quais exatamente?)

ESCOPO DAS VARIÁVEIS

- No Java, podemos declarar variáveis a qualquer momento. Porém, dependendo de onde você as declarou, ela vai valer de um determinado ponto a outro.

```
// aqui a variável i não existe  
int i = 5;  
// a partir daqui ela existe
```

- O escopo da variável é o nome dado ao trecho de código em que aquela variável existe e onde é possível acessá-la.

ESCOPO DAS VARIÁVEIS

- Quando abrimos um novo bloco com as chaves, as variáveis declaradas ali dentro só valem até o fim daquele bloco.

```
// aqui a variável i não existe
int i = 5;
// a partir daqui ela existe
while (condicao) {
    // o i ainda vale aqui
    int j = 7;
    // o j passa a existir
}
// aqui o j não existe mais, mas o i continua dentro do escopo
```

- No bloco acima, a variável `j` para de existir quando termina o bloco onde ela foi declarada.

ESCOPO DAS VARIÁVEIS

- Se você tentar acessar uma variável fora de seu escopo, ocorrerá um erro de compilação.

```
EscopoDeVariavel.java:8: cannot find symbol
symbol  : variable j
location: class EscopoDeVariavel
        System.out.println(j);
                           ^
1 error
```

ESCOPO DAS VARIÁVEIS

- O mesmo vale para um if:

```
if (algunBooleano) {  
    int i = 5;  
}  
else {  
    int i = 10;  
}  
System.out.println(i); // cuidado!
```

- Aqui a variável `i` não existe fora do `if` e do `else` !
- Se você declarar a variável antes do `if` , vai haver outro erro de compilação: dentro do `if` e do `else` a variável está sendo redeclarada!

ESCOPO DAS VARIÁVEIS

→ Então o código para compilar e fazer sentido fica:

```
int i;  
if (algumBooleano) {  
    i = 5;  
}  
else {  
    i = 10;  
}  
System.out.println(i);
```

ESCOPO DAS VARIÁVEIS

- Uma situação parecida pode ocorrer com o for :

```
for (int i = 0; i < 10; i++) {  
    System.out.println("olá!");  
}  
System.out.println(i); // cuidado!
```

- Neste for , a variável i morre ao seu término, não podendo ser acessada de fora do for, gerando um erro de compilação.
- Se você realmente quer acessar o contador depois do loop terminar, precisa de algo como:

```
int i;  
for (i = 0; i < 10; i++) {  
    System.out.println("olá!");  
}  
System.out.println(i);
```

UM BLOCO DENTRO DO OUTRO

- Um bloco também pode ser declarado dentro de outro. Isto é, um if dentro de um for, ou um for dentro de um for, algo como:

```
while (condicao) {  
    for (int i = 0; i < 10; i++) {  
        // código  
    }  
}
```

Atividade

- 1) Vimos apenas os comandos mais usados para controle de fluxo. O Java ainda possui o `do..while` e o `switch`. Pesquise sobre eles e diga quando é interessante usar cada um deles.
- 2) Algumas vezes, temos vários laços encadeados, podemos utilizar o `break` para quebrar o laço mais interno. Mas, se quisermos quebrar um laço mais externo, teremos de encadear diversos `ifs` e seu código ficará uma bagunça. O Java possui um artifício chamado `labeled loops`; pesquise sobre eles.
- 3) O que acontece se você tentar dividir um número inteiro por 0? E por 0.0? Explique.
- 4) Além dos operadores de incremento, existem os de decremento, como `--i` e `i--`. Além desses, você pode usar instruções do tipo `i += x` e `i -= x`, o que essas instruções fazem?

Dúvidas



alana.neo@ifms.edu.br